SECTION 2 · SUBSECTION 2.1
IMPLEMENTATION SEQUENCE

2.1.1 INTRODUCTION.

The implementation plan divides BaudForge into sixteen stages, eight decision gates and four release cuts. Every stage ends with a program that runs on a real board, so no stage is a report. Figure 2.1-1 gives the order; the critical path runs through the core, the GUI and the decoder.

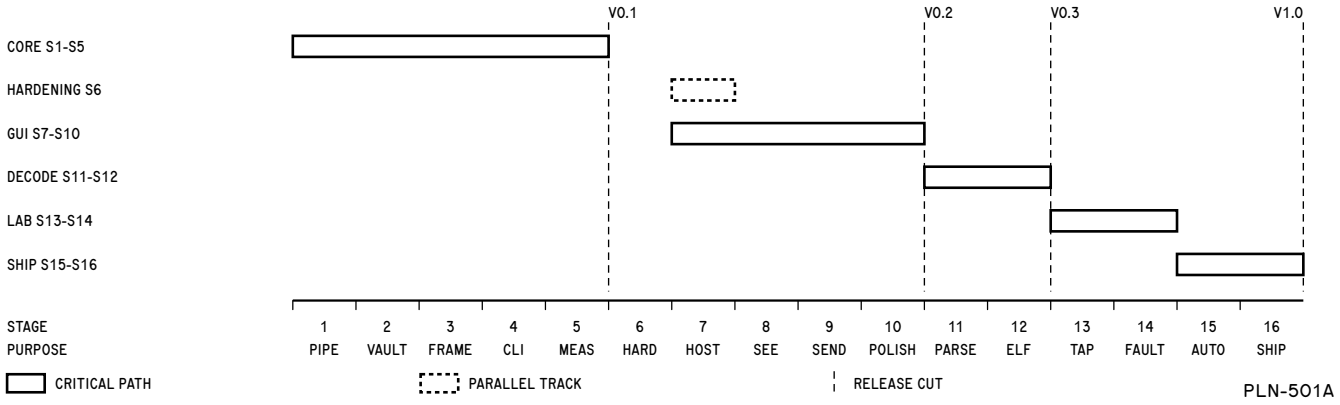


Figure 2.1-1. Stage Schedule and Release Cuts

PLN

2.1.2 FUNCTIONAL DESCRIPTION.

Work starts at S1 with the byte pipeline and the mock source. S2 adds the UART and TCP drivers with the session vault, S3 adds framing and S4 adds the CLI and the UI state contract. At S5 the plan measures two numbers and ships v0.1. After S5 the work fans into three tracks: hardening at S6, the GUI from S7 to S10, and the dissectors at S11 and S12. The tracks rejoin at S13 for interception, fault injection and release (figure 2.1-2). Eight gates guard the stage entries, and a cut ships only when its proof exists.

2.1.2.1 Gate Rule.

A gate closes on a number, not on an opinion. G0 reports the cost of each fan-out primitive, and G4 reports the transit time of a 1 MB frame window, before the stage that depends on either starts.

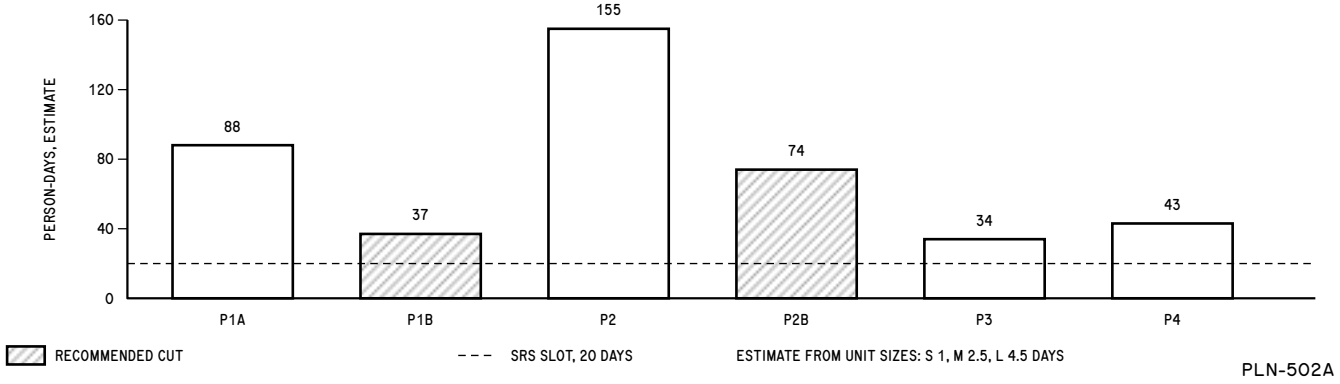


Figure 2.1-2. Estimated Effort by Phase, Against the SRS Slot

2.1.3 DECISION DATA.

Table 2.1-1. Decision Gates and Their Answer

Gate	Decision carried
GATE 0	Fan-out: two queues, one per consumer
GATE 1	Serial driver: crate first, shim if needed
GATE 2	Vault: memory map plus LZ4, both in Rust
GATE 3	Assert: TOML rules, no scripting yet
GATE 4	Frame window: raw bytes over IPC
GATE 5	Dissector: one IR per port, tree output
GATE 6	Lua 5.4, sandboxed, no file access
GATE 7	Fonts bundled, open licence, fallback

CAUTION

The SRS allows 20 person-days per milestone. Phase 1A costs 88 and Phase 2 costs 155. Ship cut B first, then the plan holds.

NEXT TEN DAYS	LEAD
1 SPEC - pipeline, overrun rule	
2 G0 - bench both primitives	
√ PICK - one queue per consumer	
3 RUN - slab pool, reader loop	
4 FAN-OUT - counters and lag	
5 SPEC - UI state contract	
6 SPEC - bflog header, chunk	